

- JS3-对象的复制
 - 1 浅拷贝
 - 1.1 展开运算符 (...)
 - 1.2 Object.assign()方法
 - 2 深拷贝
 - 2.1 递归实现
 - 2.2 structuredClone()
 - 2.3 JSON.parse(JSON.stringify(obj))
 - 3 总结
 - 3.1 深拷贝与浅拷贝的区别
 - 3.2 各方式优缺点
 - 3.3 优化建议

JS3-对象的复制

在JavaScript中，数据类型分为基本数据类型和引用数据类型。基本数据类型的值直接存储在栈内存中，而引用数据类型的实际值存储在堆内存中，栈内存中只存储指向堆内存的引用地址。（见 [JS2-数据类型-1 数据类型种类](#)）

这种存储方式导致了一个重要的现象：当我们直接复制一个引用类型时，实际上是复制了引用地址，而不是创建了一个新的对象。（这也影响了参数传递，具体见 [JS2-数据类型-4 参数传递](#)）

```
const obj = {
  a: 1,
  b: '2',
  c: [3, 4, 5],
  d: {
    e: 6
  }
}

const copy = obj; // 仅复制引用地址
console.log(copy); // { a: 1, b: '2', c: [ 3, 4, 5 ], d: { e: 6 } }

copy.a = 20;
console.log(copy.a); // 20
console.log(obj.a); // 20  可以看到原对象也被修改
```

这段代码揭示了直接赋值引用类型的问题：修改复制后的对象会同时影响原对象，因为它们指向堆内存中的同一个对象。这就是为什么我们需要了解深拷贝和浅拷贝的原因

——在某些场景下，我们需要完全独立的对象副本。

1 浅拷贝

浅拷贝是指创建一个新对象，该对象对原始对象的所有属性进行拷贝，但对于嵌套的引用类型属性，拷贝的仍然是引用地址。也就是说，**浅拷贝只复制对象的第一层属性**，对于深层嵌套的对象，新旧对象仍然共享同一份内存。

1.1 展开运算符 (...)

```
const obj = {
  a: 1,
  b: '2',
  c: [3, 4, 5],
  d: {
    e: 6
  }
}

const copy = {...obj}; // 浅拷贝-展开运算符
console.log(copy); // { a: 1, b: '2', c: [ 3, 4, 5 ], d: { e: 6 } }

copy.a = 20;
console.log(copy.a); // 20
console.log(obj.a); // 1

copy.d.e = 30;
console.log(copy.d.e); // 30
console.log(obj.d.e); // 30 深层嵌套的对象仍会被修改
```

1.2 Object.assign()方法

```
const obj = {
  a: 1,
  b: '2',
  c: [3, 4, 5],
  d: {
    e: 6
  }
}

const copy = {};
Object.assign(copy, obj); // 浅拷贝-Object.assign()方法
```

```
console.log(copy); // { a: 1, b: '2', c: [3, 4, 5 ], d: { e: 6 } }

copy.a = 20;
console.log(copy.a); // 20
console.log(obj.a); // 1

copy.d.e = 30;
console.log(copy.d.e); // 30
console.log(obj.d.e); // 30 深层嵌套的对象仍会被修改
```

通过上面的代码，我们可以清楚地看到浅拷贝的特点：

- 对于简单数据类型的属性，修改拷贝后的对象不会影响原对象
- 对于复杂数据类型的属性，修改拷贝后的对象会同时影响原对象

这就是为什么称之为"浅拷贝"——它只拷贝了对象的表层，而没有深入到嵌套对象的内部。

2 深拷贝

深拷贝是指创建一个完全独立的新对象，不仅复制对象的所有属性，还递归地复制嵌套的引用类型属性。也就是说，深拷贝会在堆内存中为所有嵌套对象创建新的副本，新旧对象之间完全独立，互不影响。

2.1 递归实现

```
function deepClone(obj = {}){
  //2. 终止条件 判断是否是值类型，值类型终止
  if(typeof obj !== "object" || obj == null){
    return obj;
  }
  //判断对象还是数组
  let result;
  if(obj instanceof Array){
    result = [];
  }else{
    result = {};
  }

  for (let key in obj) {
    console.log("循环",key,obj[key])
    if (obj.hasOwnProperty(key)) {
      //递归入口
      //1.参数：引用类型对象 返回值：值类型
      //3.单层递归逻辑 循环当前对象value中的值，赋值给新对象的key
    }
  }
}
```

```

        result[key] = deepClone(obj[key]);
    }
}
return result;
}

const obj = {
  a: 1,
  b: '2',
  c: [3, 4, 5],
  d: {
    e: 6
  }
}

const copy = deepClone(obj);    // 深拷贝-递归实现
console.log(copy);    // { a: 1, b: '2', c: [ 3, 4, 5 ], d: { e: 6 } }

copy.a = 20;
console.log(copy.a);    // 20
console.log(obj.a);    // 1

copy.d.e = 30;
console.log(copy.d.e);    // 30
console.log(obj.d.e);    // 6

```

2.2 structuredClone()

`structuredClone()` 是ES2022新增的深拷贝方法，用法简单，能处理大部分引用类型（对象、数组、Map、Set、Date、RegExp、ArrayBuffer等），支持BigInt类型，且能正确拷贝循环引用（比如`obj.self = obj`，不会报错）。

注意：`structuredClone()` 有以下局限：

- 无法拷贝函数（普通函数、箭头函数、类方法均会被忽略）；
- 无法拷贝原型链上的属性（只拷贝对象自身属性）；
- 无法拷贝Error、Function、GeneratorFunction、AsyncFunction等类型；
- 无法拷贝Symbol类型（会被忽略）；
- 无法拷贝DOM节点（会报错）。

```

const obj = {
  a: 1,
  b: '2',
  c: [3, 4, 5],
  d: {
    e: 6
  }
}

```

```
const copy = structuredClone(obj);    // 深拷贝-structuredClone
console.log(copy);    // { a: 1, b: '2', c: [ 3, 4, 5 ], d: { e: 6 } }

copy.a = 20;
console.log(copy.a);    // 20
console.log(obj.a);    // 1

copy.d.e = 30;
console.log(copy.d.e);    // 30
console.log(obj.d.e);    // 6
```

2.3 JSON.parse(JSON.stringify(obj))

```
const obj = {
  a: 1,
  b: '2',
  c: [3, 4, 5],
  d: {
    e: 6
  }
}

const copy = JSON.parse(JSON.stringify(obj));    // 深拷贝-JSON.parse JSON.stringify
console.log(copy);    // { a: 1, b: '2', c: [ 3, 4, 5 ], d: { e: 6 } }

copy.a = 20;
console.log(copy.a);    // 20
console.log(obj.a);    // 1

copy.d.e = 30;
console.log(copy.d.e);    // 30
console.log(obj.d.e);    // 6
```

注意：这个方法有明显局限，无法处理以下类型，会自动忽略或转换：

- 函数（被忽略）
- BigInt（会报错，无法转换）
- undefined（被忽略）
- NaN、Infinity（会转为null）

3 总结

3.1 深拷贝与浅拷贝的区别

特点	浅拷贝	深拷贝
复制层级	仅复制对象的第一层属性	递归复制对象的所有层级属性
引用类型	对于嵌套的引用类型，复制的是引用地址	对于嵌套的引用类型，创建新的对象副本
相互影响	修改复制对象的嵌套属性会影响原对象	修改复制对象不会影响原对象
实现复杂度	相对简单	相对复杂
性能消耗	较低	较高
应用场景	对象结构简单，没有嵌套的引用类型 性能要求较高，不需要完全独立的对象副本 只需要修改对象的第一层属性	对象结构复杂，包含多层嵌套的引用类型 需要完全独立的对象副本，修改复制对象不能影响原对象 数据持久化或状态管理场景

3.2 各方式优缺点

实现方式	优点	缺点
展开运算符 (...)	语法简洁，使用方便	只能处理可迭代对象，对于某些特殊对象可能不适用
Object.assign()	兼容性较好，使用广泛	只复制可枚举的自有属性，不复制继承的属性
递归实现	可以根据需要定制拷贝逻辑	实现复杂，需要处理循环引用等边界情况
structuredClone()	见2.2	见2.2
JSON.parse(JSON.stringify(obj))	实现简单，无需额外依赖	见2.3

3.3 优化建议

1. **选择合适的复制方式**：根据实际需求选择浅拷贝或深拷贝，避免不必要的性能消耗
2. **处理边界情况**：在实现自定义深拷贝函数时，注意处理循环引用、特殊对象类型等边界情况
3. **使用成熟的工具库**：在实际项目中，优先使用Lodash等成熟的工具库来实现深拷贝，以确保代码的稳定性和可靠性
4. **性能优化**：对于大型复杂对象，可以考虑使用更高效的深拷贝算法，或使用惰性加载等技术来减少性能消耗